Designøving 3

Tittel: Sekvensgjenkjenning med VHDL	
Forfattere: Mathias Espelund Ekse	
Versjon: 1.0	Dato: 14.10.2026

Innhold

1 Oppdrag **2**

2 Implementasjon i VHDL **2**

2.1 Blokkdiagram 2

2.2 Systemkode 3

2.3 Kompilasjonsoppsummering 5

2.4 RTL-diagram 6

3 Simulasjon og resultater **6**

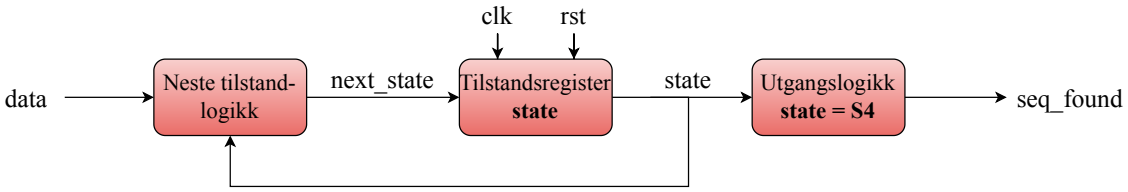
3.1 Simulasjon 6

1 Oppdrag

Oppdraget er å designe og implementere en sekvensdetektor i VHDL som overvåker en bitstrøm på inngangen `data` og setter utgangen `seq_found` høy i én klokkesyklus når sekvensen `1011` er mottatt. Enden på én sekvens kan være starten på neste. Systemet er synkront med klokken `clk` og har aktivt høy reset `rst`. Designet løses først som en tilstandsmaskin (FSM), og deretter som et skiftregister med kombinatorisk logikk (ekstraoppdrag). Begge varianter implementeres som hver sin arkitektur til samme entitet, `d3_sequence_detector`, og testes med den vedlagte testbenken.

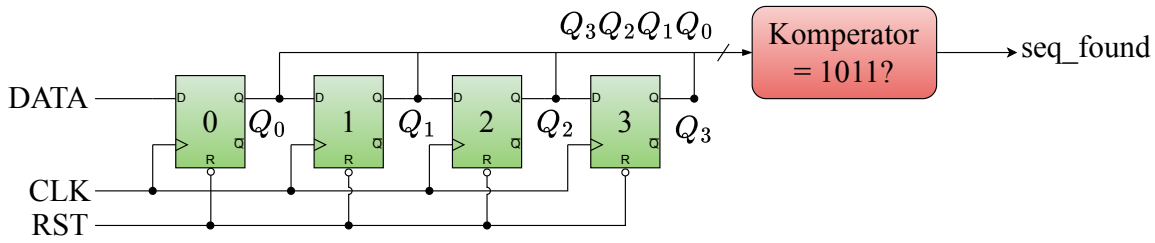
2 Implementasjon i VHDL

2.1 Blokkdiagram



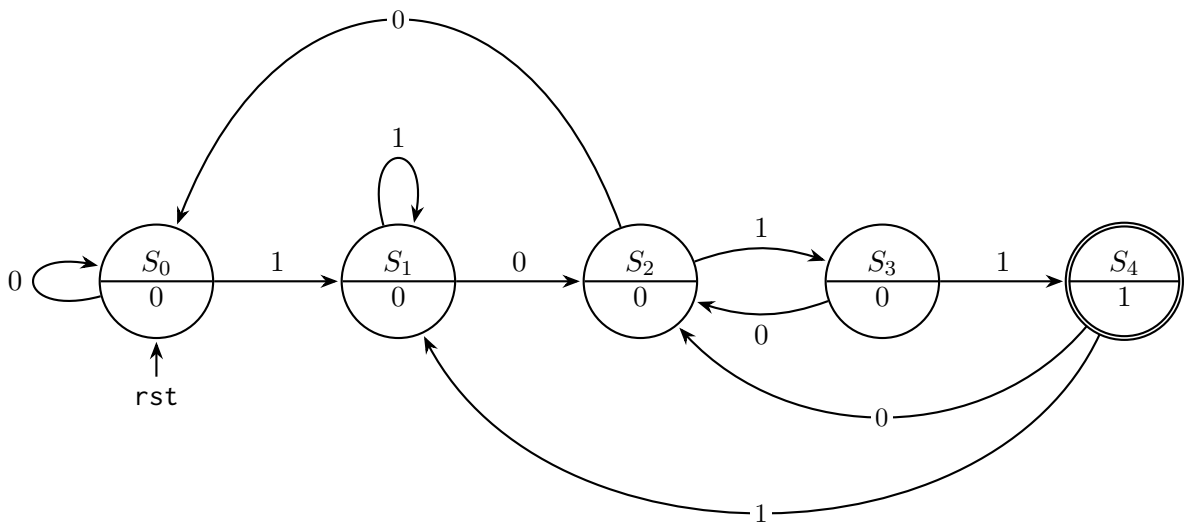
Figur 1: Blokkdiagram for den FSM-baserte sekvensdetektoren.

Figur 1 viser blokkdiagrammet for den FSM-baserte sekvensdetektoren. Inngangssignalet `data` brukes sammen med den nåværende tilstanden til å bestemme neste tilstand. Tilstandsregisteret oppdateres på stigende flanke av klokkesignalet `clk`, mens `rst` setter systemet tilbake til starttilstanden S0. Utgangen `seq_found` blir høy når systemet befinner seg i tilstand S4, som tilsvarer at sekvensen 1011 er oppdaget.



Figur 2: Blokkdiagram for sekvensdetektoren implementert med skiftregister.

Figur 2 viser løsningen basert på et skiftregister. Hvert innkommende bit forskyver registerinnholdet én posisjon på hver stigende klokkeflanke. Innholdet sammenlignes kontinuerlig med bitmønsteret 1011, og utgangen blir høy når mønstrene samsvarer.



Figur 3: Tilstandsdiagram for FSM-basert deteksjon av sekvensen `1011`.

Figur 3 viser tilstandsdiagrammet for sekvensdetektoren. Tilstandene S0–S3 representerer hvor mye av sekvensen 1011 som er gjenkjent. Når det siste bitet mottas i tilstand S3, går systemet til S4, og `seq_found` blir høy. Overgangene er utformet slik at deler av en tidligere sekvens kan brukes videre dersom sekvenser overlapper. Ubrukte tilstandskoder fører tilbake til S0.

2.2 Systemkode

Koden i kodeutdrag 1 viser VHDL-koden for sekvensdetektoren med begge arkitekturene. `fsm_rtl` implementerer tilstandsmaskinen med to prosesser: en sekvensiell prosess som oppdaterer tilstanden på stigende klokkeflanke (med asynkron reset), og en kombinatorisk prosess som velger neste tilstand ut fra nåværende tilstand og `data`. Ubrukte tilstandskoder håndteres med `when others`, som sender maskinen til S_0. Arkitekturen `sr_rtl` implementerer skiftregisteret og sammenligner innholdet med konstanten `1011`.

Kodeutdrag 1: d3_sequence_detector.vhd

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity d3_sequence_detector is
  -- Setter clk, rst og data som innganger, seq_found som utgang
  port(
    clk       : in std_logic;
    rst       : in std_logic;
    data      : in std_logic;
    seq_found : out std_logic
  );
end entity d3_sequence_detector;


------------------------------------------------------------
-- ARKITEKTUR FOR TILSTANDSMASKIN
```

```vhdl
------------------------------------------------------------
architecture fsm_rtl of d3_sequence_detector is
 -- Tilegner hver av de 5 tilstandene en 3-bits kode
 -- Bruker std_logic_vector for å bestemme utfallet av de ubrukte kodene 101, 110 og
    ↪ 111, som går til S0 via when others.
 subtype t_state is std_logic_vector(2 downto 0);

 constant S0 : t_state := "000";  -- Ingen mønster sett
 constant S1 : t_state := "001";  -- Sett '1'
 constant S2 : t_state := "010";  -- Sett "10"
 constant S3 : t_state := "011";  -- Sett "101"
 constant S4 : t_state := "100";  -- Sett "1011" (altså funnet sekvens)

 signal state      : t_state := S0;     -- Nåværende tilstand
 signal next_state : t_state;           -- Neste tilstand

begin
 -- Sekvensiell prosess som sjekker tilstand
 -- Sjekk av asynkron reset, deretter oppdatering på klokkens stigende flanke
 p_state: process(clk, rst)
 begin
   if rst = '1' then
     state <= S0;
   elsif rising_edge(clk) then
     state <= next_state;          -- State endres til valgt neste tilstand
   end if;
 end process p_state;


 -- Kombinatorisk prosess som velger neste tilstand basert på data
 p_next_state : process(state, data)
 begin
   next_state <= S0;

   case state is
     when S0 =>
       if data = '1' then
         next_state <= S1;
       else
         next_state <= S0;
       end if;
     when S1 =>
       if data = '1' then
         next_state <= S1;
       else
         next_state <= S2;
       end if;
     when S2 =>
       if data = '1' then
         next_state <= S3;
       else
         next_state <= S0;
       end if;
     when S3 =>
       if data = '1' then
         next_state <= S4;
       else
```

```
          next_state <= S2;
        end if;
      when S4 =>
        if data = '1' then
          next_state <= S1;
        else
          next_state <= S2;
        end if;
      when others =>
        next_state <= S0;
    end case;


 end process p_next_state;
 -- Setter til slutt seq_found = 1 dersom bitmønster '1011' oppdages.
 seq_found <= '1' when state = S4 else '0';

end architecture fsm_rtl;


-------------------------------------------------------
-- ARKITEKTUR FOR SKIFTREGISTER
-------------------------------------------------------
architecture sr_rtl of d3_sequence_detector is
  constant correct_sequence : std_logic_vector(3 downto 0) := "1011";
  signal data_sequence      : std_logic_vector(3 downto 0) := "0000";

begin

  p_shift_register : process(clk, rst)
  begin
    if rst = '1' then
      data_sequence <= "0000";

    elsif rising_edge(clk) then
      data_sequence <= data_sequence(2 downto 0) & data;
    end if;

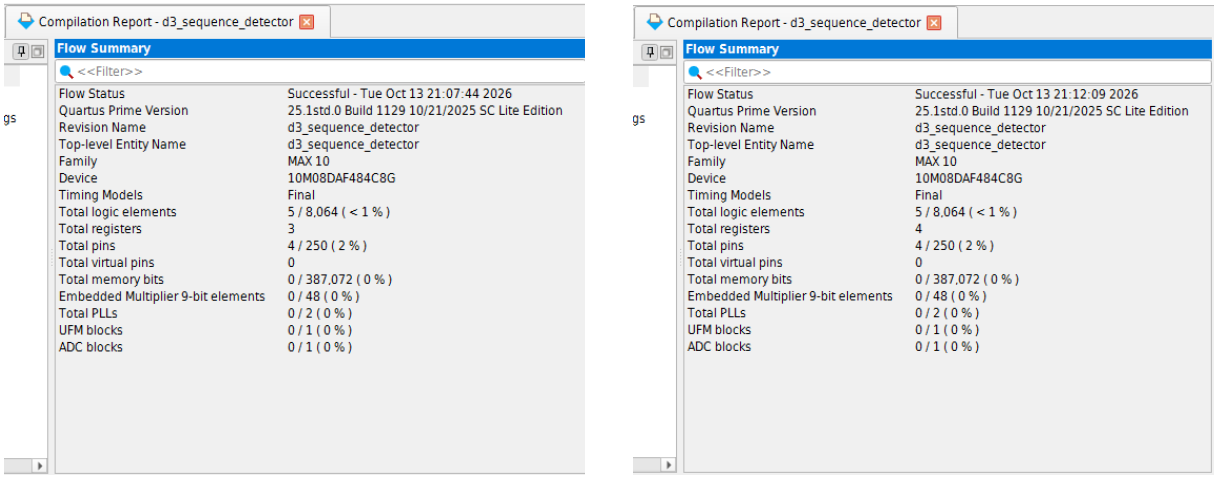
  end process p_shift_register;

  seq_found <= '1' when data_sequence = correct_sequence else '0';

end architecture sr_rtl;
```

2.3 Kompilasjonsoppsummering

Kompilasjonsoppsummeringene fra Quartus Prime Lite er vist i figur 4. Begge variantene bruker 5 logikkelementer og 4 pinner. FSM-varianten bruker 3 registre (tilstandsvektoren på 3 bit), mens skiftregister-varianten bruker 4 registre (ett per bit i skiftregisteret).

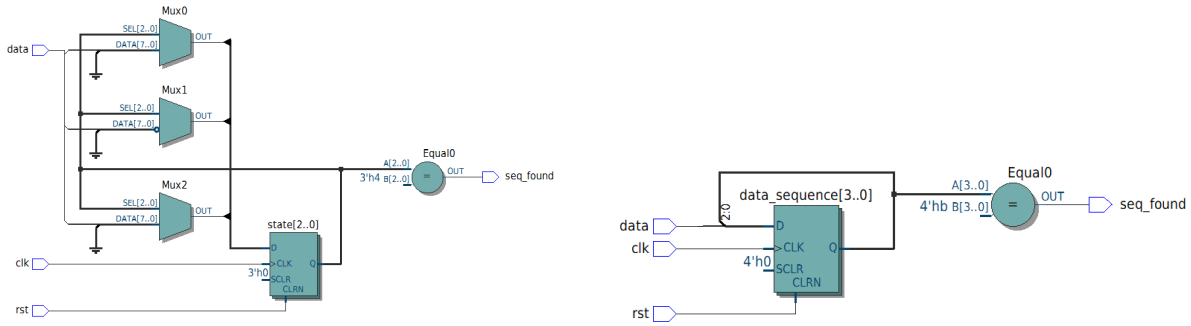


(a) FSM-variant **(b)** Shift-Register variant

Figur 4: Kompilasjonsoppsummering fra Quartus Prime Lite for både FSM- og skiftregistervarianten.

2.4 RTL-diagram

RTL-diagrammene er vist i figur 5. FSM-varianten (figur 5a) består av et tilstandsregister `state[2..0]`, tre multipleksere som velger neste tilstand ut fra `data`, og en komparator som setter `seq_found` når tilstanden er `100` (S_4). Skiftregister-varianten (figur 5b) består av et 4-bits register `data_sequence[3..0]` og en komparator mot `1011`.



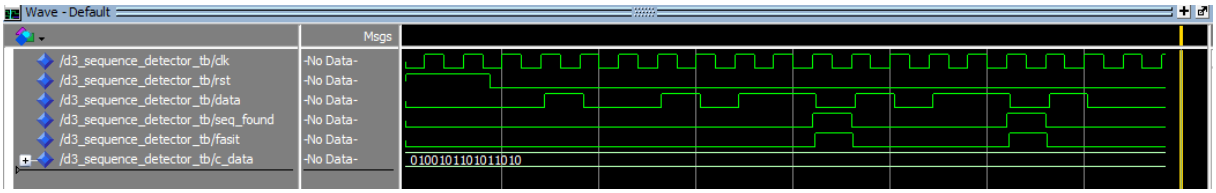
(a) FSM-variant **(b)** Shift-Register variant

Figur 5: RTL-diagrammer fra Quartus Prime for FSM- og skiftregistervarianten.

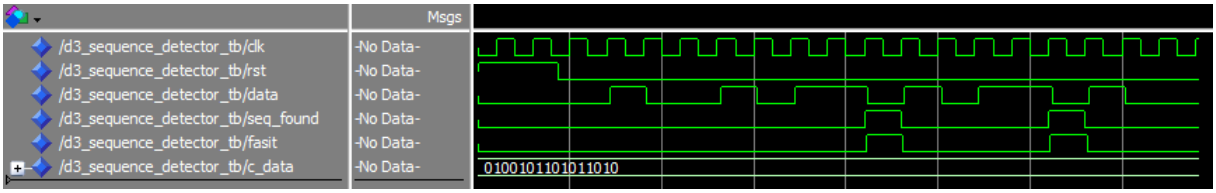
3 Simulasjon og resultater

3.1 Simulasjon

Testbenken ble simulert i ModelSim, og resultatet er vist i figur 6 og figur 7.



Figur 6: Simuleringsresultat i ModelSim for FSM-varianten



Figur 7: Simuleringsresultat i ModelSim for skiftregistervarianten

Figurene viser simuleringsresultatene fra ModelSim for de to implementasjonene. Testbenken sender inn bitstrømmen `0100101101011010` på signalet `c_data`. Utgangen `seq_found` blir høy når sekvensen `1011` oppdages. Dette skjer to ganger i bitstrømmen, og utgangen går høy i én klokkesyklus etter hver deteksjon. Simuleringen viser at begge implementasjonene gir forventet resultat (sammenligner mot fasit).